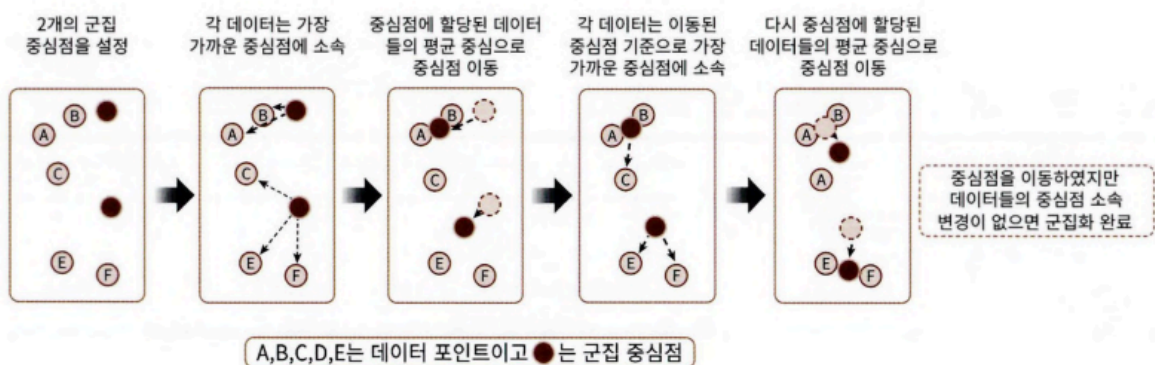


파머완 7장 - 군집화

01. K-means 알고리즘



중심점 (centroid)라는 특정한 임의의 지점을 선택해 해당 중심에 가장 가까운 포인트들을 선택하는 군집화 기법



1 K-means 알고리즘 동작 과정

Step 1

k개의 centroid 위치 random 설정 (초기화 알고리즘 - 적합한 위치)

Step 2

데이터들은 가장 가까운 centroid에 소속

Step 3

각 centroid들을 소속된 데이터들의 평균 중심으로 이동

Step 4

변경된 중심에 맞게 각 데이터들을 다시 군집화

Step 5

다시 중심을 변경된 데이터들의 평균 중심으로 이동

Step 6

Step 4로 이동, 데이터들의 중심점 소속 변경이 없을 때까지 반복

2 K-means 장점

1. clustering에서 가장 많이 활용됨
2. 알고리즘이 쉽고 간결함
3. 개별 군집 내 데이터가 원형으로 흩어져 있을 때 효과적

3 K-means 단점

1. 거리기반 알고리즘 → feature수가 많을 수록 정확도 ↓ (PCA로 차원감소)
2. 반복 횟수가 많을수록 수행 시간 ↑
3. k 설정 어려움

4 Scikit-learn KMeans 클래스

```
class sklearn.cluster.KMeans(n_clusters=8, init='k-means++', n_init=10, max_iter=1000, tol=0.0001, precompute_distances='auto', verbose=0, copy_x=True, n_jobs=1, algorithm='auto')
```

중요 파라미터

- n_clusters : k(centroid의 개수)
- init : centroid 설정 방식
- max_iter : 최대 반복 횟수
max_iter전에 모든 데이터의 중심점 이동이 없으면 종료

수행 방법

- fit(데이터셋) / fit_transform(데이터셋)

주요 속성

- kmeans객체.labels_ : centroid 레이블값

- `kmeans` 객체.`cluster_centers_`: centroid 좌표 (shape - (k수, feature수))
시각화할 때 사용 가능

02. 군집 평가(Cluster Evaluation)

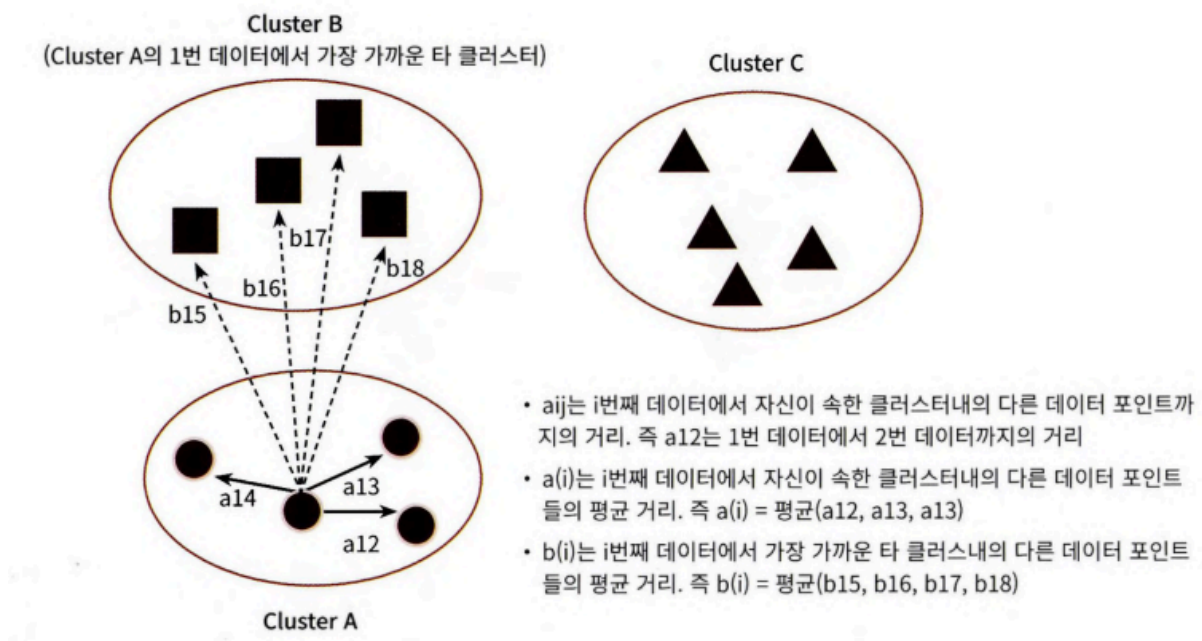
- 대부분의 군집화 데이터셋은 비교할 만한 정답(target label)이 없음
- `classification`과 달리 데이터 내 숨어있는 별도의 그룹 찾아서 의미 부여
or 동일한 군집에 속해도 그 안에서 더 세분화된 군집화 추구
or 다른 군비에 속해도 더 넓은 군집화 레벨화
- 군집화가 잘됐는지 어떻게 평가? > 실루엣 분석(silhouette analysis)

1 실루엣 분석 (silhouette analysis)



군집내 거리 ↓, 군집간 거리 ↑ 잘 되어있는지 계수로 나타냄.
개별 데이터가 가지는 실루엣 계수

→ 해당 데이터가 같은 군집 내 데이터와 얼마나 가까운지/
다른 군집 데이터와 얼마나 멀리 분리되어있는지 나타냄



2 실루엣 계수



개별 데이터가 가지는 실루엣 계수

→ 해당 데이터가 같은 군집 내 데이터와 얼마나 가까운지/
다른 군집 데이터와 얼마나 멀리 분리되어있는지 나타냄

$$s(i) = \frac{(b(i) - a(i))}{(\max(a(i), b(i)))}$$

- $b(i)$: 해당 데이터가 속하지 않은 군집 중 가장 가까운 군집과의 평균 거리
- $a(i)$: 해당 데이터가 속한 군집내 다른 데이터들과의 평균 거리
- $b(i) - a(i)$: 두 군집 간의 거리가 얼마나 떨어져 있어?
- $/(\max(a(i), b(i)))$: 정규화 >> $[-1, 1]$ 의 값을 가짐.
- 1에 가까울수록 군집간 거리 
- 0에 가까울수록 군집간 거리 
- 값은 다른 군집에 데이터가 할당됨.

3 scikit-learn에서의 실루엣 분석 메소드

```
sklearn.metrics.silhouette_samples(X, labels, metric='euclidean', **kwargs)
```

- 인자 : X - feature dataset,
labels - 각 feature가 속한 군집 라벨값
- 반환 : 각 데이터 포인트의 실루엣 계수

```
sklearn.metrics.silhouette_score(X, labels, metric='euclidean' sample_size = N)
```

- 인자 : X - feature dataset,
labels - 각 feature가 속한 군집 라벨값
- 반환 : 전체 데이터의 실루엣 계수의 평균값
= `np.mean( silhouette_samples( ) )`
0 ~ 1 사이값
- 이 값이 높을수록 군집화가 잘됐다고 판단 가능(무조건은 아님)

4 좋은 군집화의 기준

1. 전체 실루엣 계수의 평균값 (= `sklearn.metrics.silhouette_score()`) 1에 가까울수록 좋음
2. 개별 군집의 평균값 / 전체 실루엣 계수의 평균값의 편차가 크지 않아야 함
= 개별 군집의 실루엣 계수 평균값이 전체 실루엣 계수 평균값에서 크게 벗어나지 **✗**
ex) 특정 군집 실루엣 계수 평균값 **↑**, 다른 군집 실루엣 계수 평균값 **↓** → 좋은 군집화 **✗**

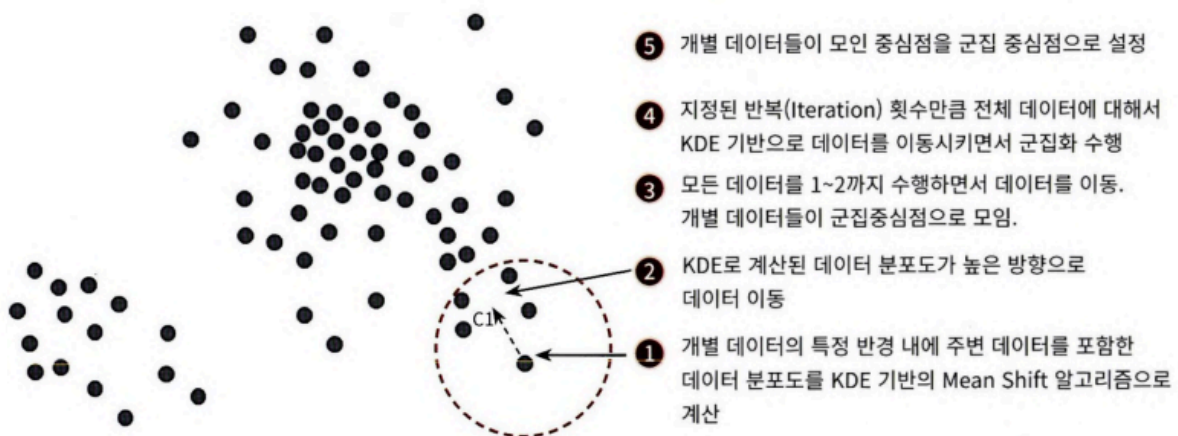
03. 평균 이동(Mean Shift)



centroid를 데이터가 모여있는 / 밀도가 가장 높은 곳으로 이동
(K-means와의 차이 : centroid 이동 방식)

밀도(도수)가 가장 높은 곳 >> pdf (probability density function)의 peak

1 평균 이동 군집 과정



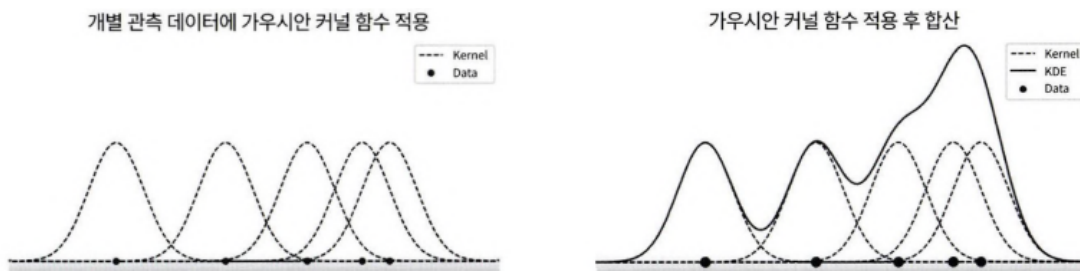
- centroid = pdf 의 peak
- pdf 를 찾기 위해 KDE(Kernel Density Estimation) 이용.

- 입력값 : 주변 데이터와의 거리값 → KDE 함수 → 반환값 → 현재 위치에서 업데이트

2 KDE (Kernel Density Estimation)



Kernel (커널) 함수를 통해 어떤 변수의 확률밀도함수 (pdf) 추정
→ pdf를 통해 변수의 특성 (기술통계값), 확률 분포 등을 알 수 있음.



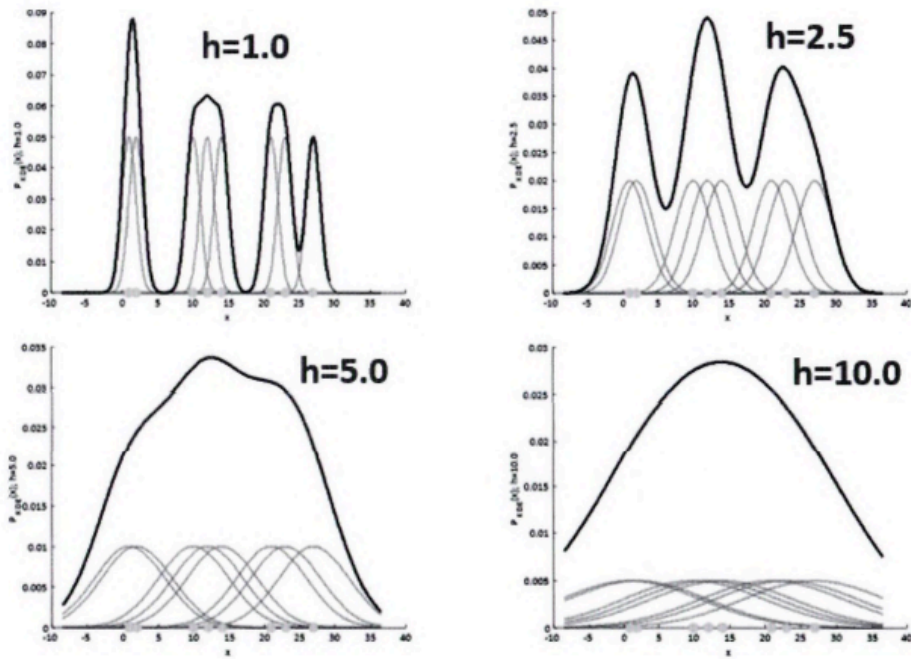
- 관측 데이터 각각에 커널함수 적용 → 모두 더한 후 데이터 수(n)으로 나눔
- 대표적인 커널 함수 : 가우시안 분포 함수

KDE의 수식

$$\text{KDE} = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- K : 커널 함수
- x : 확률 변수
- h : 대역폭 (bandwidth)

3 KDE의 대역폭 (bandwidth)



- KDE 형태 - 부드럽게/뾰족하게 smoothing 하는데 사용, (⚠ kurtosis와는 다른 개념)
- 적당한 h값을 계산하는 것은 KDE 기반 평균 이동 군집화에서 매우 중요함.

h ↓ (1.0)

- 좁고 뾰족한 KDE
- 변동성 ↑ → over-fitting
- 일반적으로 대역폭이 작을수록 centroid 수 ↑

h ↑ (10)

- 과도하게 smoothing(평활화)된 KDE
- 지나치게 단순화 → under-fitting
- 일반적으로 대역폭이 클수록 centroid 수 ↓

4 scikit-learn에서의 평균 이동 클래스

1. MeanShift 클래스

```
class sklearn.cluster.MeanShift(*, bandwidth=None, seeds=None, bin_seeding=False,
                                min_bin_freq=1, cluster_all=True,
                                n_jobs=None, max_iter=300)
```

2. estimate_bandwidth 함수 - 적의 대역폭 계산

```
sklearn.cluster.estimate_bandwidth(X, *, quantile=0.3, n_samples=None,
                                   random_state=0, n_jobs=None)
```

5 Mean Shift의 장점

1. 이상치 영향 ↓
2. 유연한 군집화 (데이터셋 특정 형태/분포도 기반으로 가정 ✗)
3. 미리 군집 개수 설정 ✗

6 Mean Shift의 단점

1. 수행시간 ↑
2. band-width(대역폭)이 군집화에 미치는 영향 ↑

→ 분석 업무 기반 데이터셋보다는 컴퓨터 비전 영역에서 사용 多,
이미지/영상 데이터 에서 특정 개체 구분/ 움직임 추적에 뛰어남

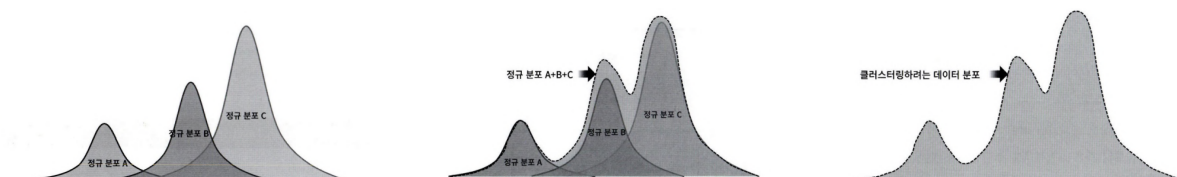
04. GMM(Gaussian Mixture Model)

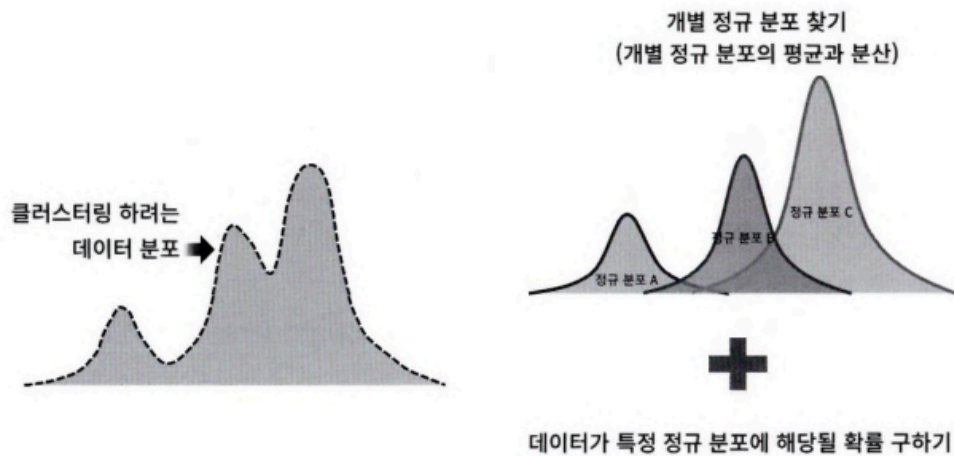


데이터가 여러개의 정규분포를 가진 데이터 집합들이 섞여 생성된 것이라는 가정 하에 군집화 수행

k-means : 거리 기반 , GMM : 확률 기반

1 GMM 과정





- 서로 다른 정규 분포에 기반해 군집화 수행
- ex) 1000개 데이터셋 > 여러개의 정규 분포 추출
> 개별 데이터가 이중 어느 정규 분포에 속하는지 결정
- 모수 추정
 - 개별 정규 분포의 평균, 분산
 - 각 데이터가 어떤 정규분포에 해당할지 확률
- EM (Expectation and Maximization)
 - 모수 추정하는데 적용
- kmeans와 달리 원형이 아닌 데이터셋에서도 유연하게 적용됨.
- 수행 시간이 오래 걸림.

2 scikit-learn에서의 GMM 클래스

```
class sklearn.mixture.GaussianMixture(n_components=1, *, covariance_type='
    reg_covar=1e-06, max_iter=100, n_ini
    init_params='kmeans', weights_init=N
    precisions_init=None, random_state=
    warm_start=False, verbose=0, verbo:
```

주요 파라미터

- n_components : Gaussian Mixture의 Model의 총 개수(K-means의 n_clusters)

05. DBSCAN (Density Based Spatial Clustering of Applications with Noise)



분포가 복잡한 데이터셋도 군집화 가능

예) 도넛 2개 짜리 데이터 세트 - kmeans, gmm, mean shift로는 군집화 안됨

1 DBSCAN 주요 파라미터

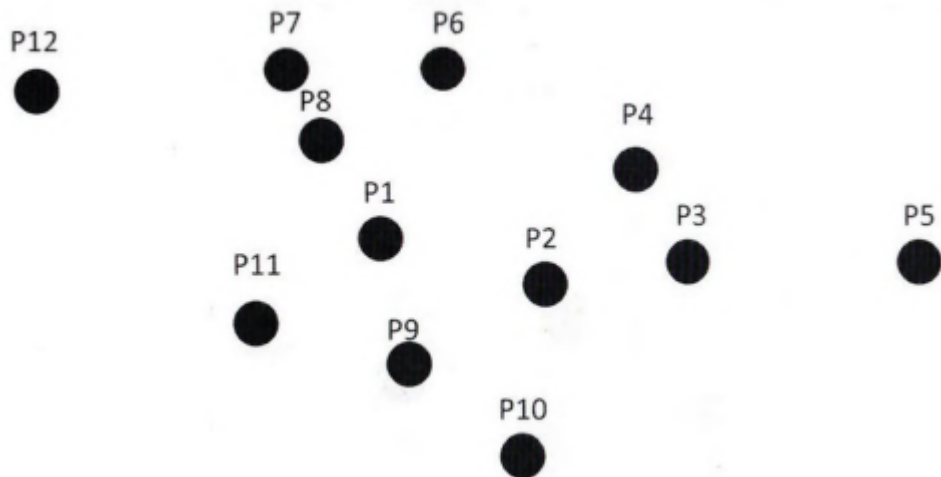
1. 입실론 주변 영역 (epsilon) - 개별 데이터 중심으로 입실론 반경을 가지는 원형의 영역
2. 최소 데이터 개수 (min points) - 입실론 주변 영역에 포함되는 타 데이터의 개수

입실론 주변 영역 내 최소 데이터의 개수 충족시키는데 따른 데이터 포인트 정의

1. 핵심 포인트(Core Point) : 주변 영역 내 min points 이상 타 데이터 有 → 해당 데이터
2. 이웃 포인트 (Neighbor Point) : 주변 영역 내 위치한 타 데이터
3. 경계 포인트 (Border Point) : 주변 영역 내 min points 이상 이웃 포인트 無,
핵심 포인트를 이웃 포인트로 가지는 데이터
4. 잡음 포인트 (Noise Point) : min points 이상의 이웃 포인트 無,
핵심 포인트를 이웃 포인트로 가지지 않는 데이터

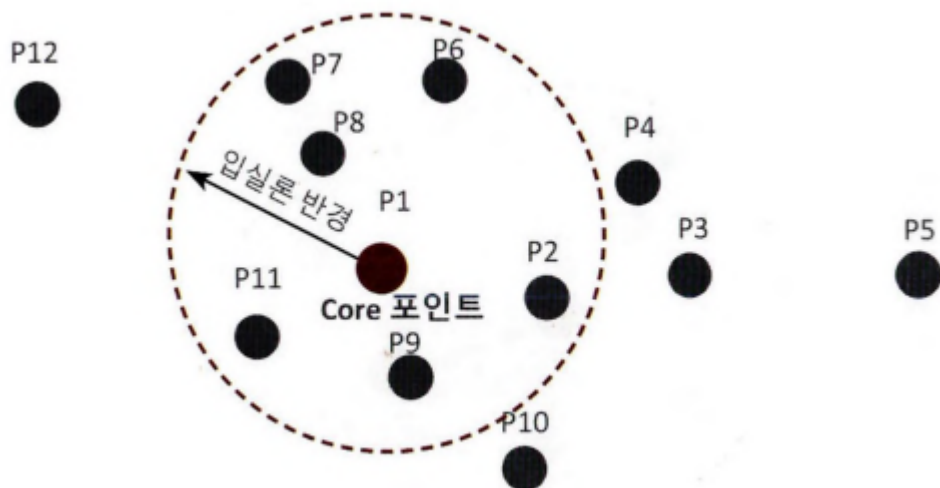
2 DBSCAN 과정

Step 0. 대상 데이터셋

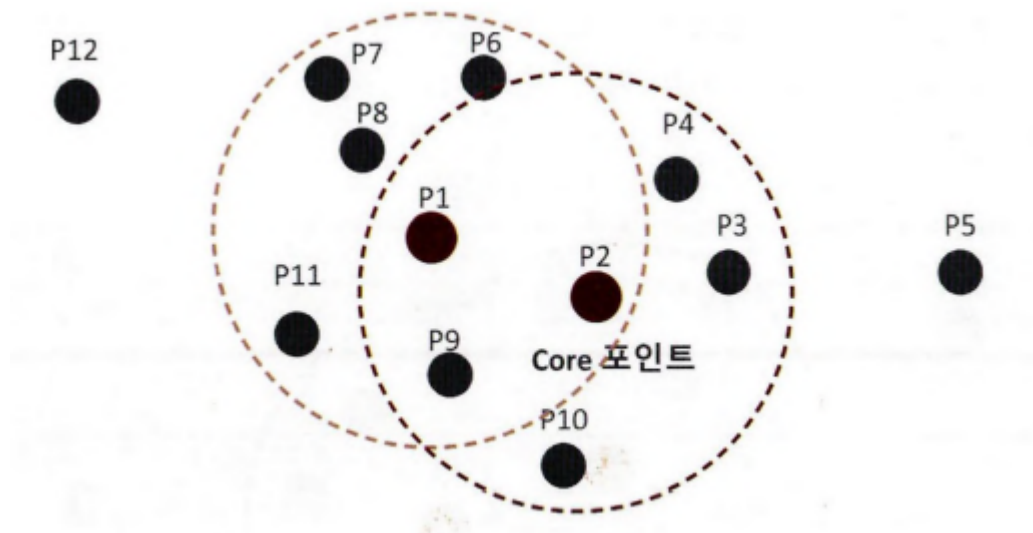


Step 2.

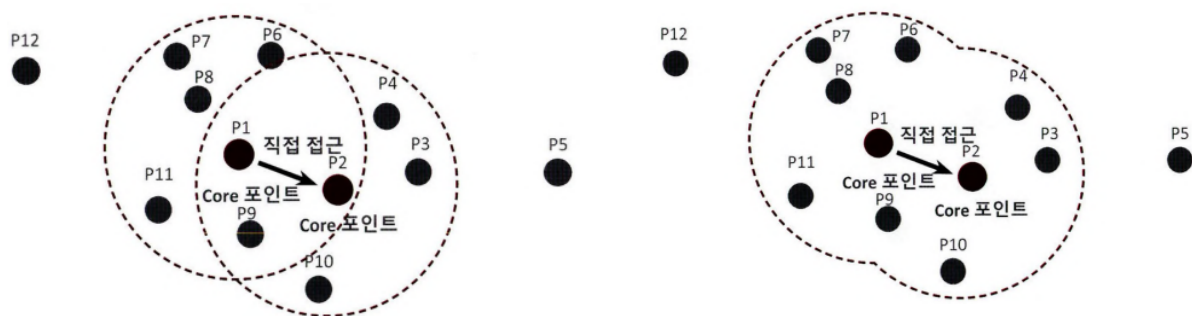
: P1을 기준으로 epsilon 반경 내 포함 데이터 7개 (> minpoints=5)
 → P1 = 핵심 포인트 (Core point)



#Step 3. P2도 같은 원리로 핵심 포인트(Core Point)

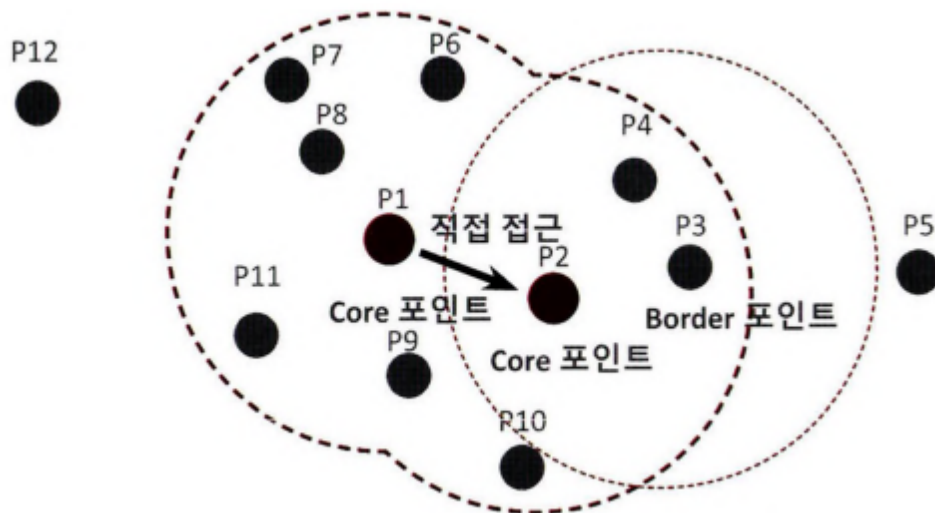


Step 4. 핵심 포인트끼리 이웃 데이터 포인트일 경우 > 직접 접근해서 cluster 합체



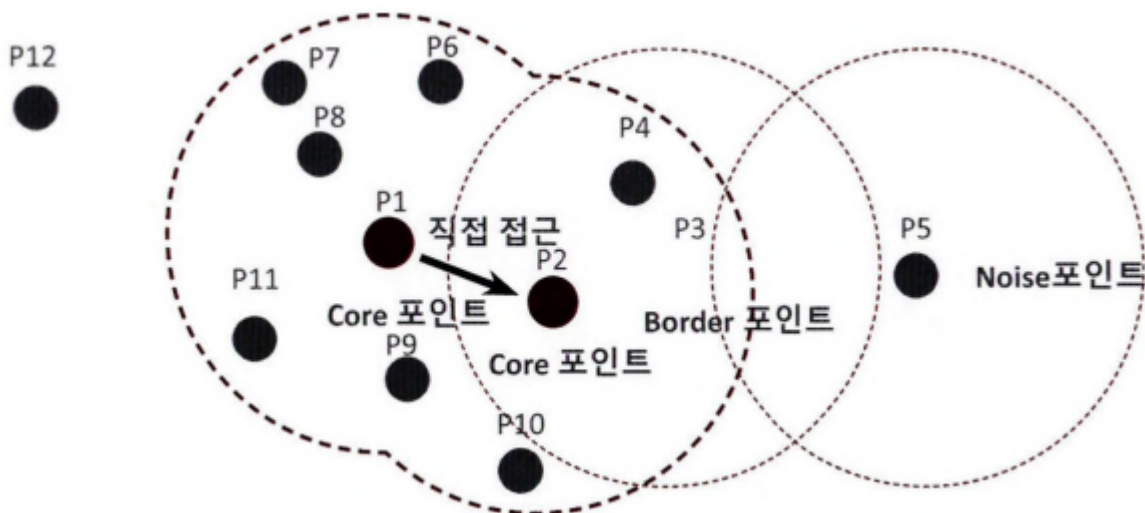
Step 5. P3은 핵심포인트가 아니지만 핵심 포인트인 P2를 이웃 포인트로 가짐.

→ P3는 경계 포인트(Border Point), 군집 외곽 형성



Step 6. P5는 핵심포인트도 아니고 핵심포인트를 이웃포인트로 가지지 않음.

→ P5는 잡음 포인트(Noise Point)



3 scikit-learn에서의 DBSCAN 클래스

```
class sklearn.cluster.DBSCAN(eps=0.5, *, min_samples=5, metric='euclidean',
                              metric_params=None, algorithm='auto', leaf.
                              p=None, n_jobs=None)
```

주요 파라미터

- eps : 입실론 주변 영역의 반경
일반적으로 1 이하의 값 설정

- eps  : Noise Point 
- min_samples : min points + 1,
자기 자신을 포함하는, 핵심포인트가 되기 위한 영역 내 데이터 최소
의 개수
 - min_samples  : Noise Point 

06. 군집화 실습 - 고객 세그먼테이션

** 진행한 실습 내용은 colab에 마크다운 형태로 정리되어있습니다.